

מרחב המצבים

אנחנו מדברים על עולם אקספוננציאלי - מחשב לא יכול להכיל את כל מרחב המצבים.

הגדרה: State Space הוא אוסף כל מצבי העולם האפשריים.

בבעיית חיפוש יש לנו:

- מצב התחלתי s_0
- פעולות אפשרויות עבור כל מצב
- פונקציית מעבר $s' = T(s, a)$
- פרידיקט מטרה $Goal(s)$
- לפעמים - מחיר של פעולה או של מצב

דוגמה יש לנו 3 סטודנטים, A, B, C . כל סטודנט יכול להיות באחד משני מצבים, מיוצגים ע"י ביט אחד:

- 0 = לא באוניברסיטה
- 1 = כן באוניברסיטה

הפעולות שלנו הן:

- $TurnOn(X)$ - מכניס את הסטודנט X לאוניברסיטה (מדליק את הביט שלו)
- $TurnOff(X)$ - מוציא את הסטודנט X מהאוניברסיטה (מכבה את הביט שלו)

עבור 3 סטודנטים יש לנו $2^3 = 8$ מצבים. אבל עבור n סטודנטים, יהיו לנו 2^n מצבים - כלומר זה גדל באופן אקספוננציאלי.

אי לרוב הבעיות מרחב המצבים הוא עצום - לא נוכל להריץ BFS, DFS, או דייקסטרה.

שאלות חיפוש ב-AI (Planning Perspective)

AI לרוב משתמשים בחיפוש כדי לפתור בעיות תכנון. שאלים:

- האם קיים בכלל מסלול מ s_0 למטרה?
- מה המסלול מ s_0 למצב מטרה כלשהו?
- מה המסלול האופטימלי מ s_0 למצב מטרה כלשהו?

שאלות מנחות:

- האם הגדרתי נכון את המטרות ומצבי המטרה?
- האם הגדרתי נכון את המצבים?

• האם הגדרתי את כל הפעולות האפשריות?

• האם אפשר לייצג מצבים לא חוקיים?

• האם המעברים שלי דטרמיניסטים או סטוכסטים?

• האם המחיר אחיד?

• האם אפשר להימנע מכניסה חוזרת לאותם מצבים?

בקורס הזה אנחנו רוצים אלגוריתמים טובים - אבל לא בהכרח אופטימלים. לא בהכרח קיים אלגוריתם שמוצא את המסלול האופטימלי בצורה יעילה. אנחנו מחפשים tradeoff בין משאבים לבין איכות הפיתרון.

הגדרות לאלגוריתם

הגדרה: אלגוריתם הוא complete אם:

- אם קיים פתרון הוא יחזיר אותו.
- אם לא קיים פתרון הוא יחזיר כישלון

הגדרה: אלגוריתם הוא אופטימלי אם הוא complete - וגם מחזיר את הפתרון האופטימלי (בהינתן שיש פתרון)

הגדרה: חיפוש סיסטמטי: מחפש בכל המרחב, מפתח את כל המצבים.

הגדרה: חיפוש לוקאלי: בודק בכל פעם חלק מהמצבים, לפי תנאים מסויימים.

הגדרה: Uninformed Search - אלגוריתם שלא משתמש ביוריסטיקות.

ייצוג עץ החיפוש

יש לנו Search Graph שבו כל קודקוד מחזיק:

- מצב עולם
- הורה - הקודקוד בעץ שממנו הגענו למצב הזה
- פעולה - הפעולה שהביא אותנו מההורה למצב העולם
- מחיר - כמה עלו הפעולות מהמצב ההתחלתי עד לכאן

אלגוריתמים סיסטמטיים

BFS Breadth First Search. מפתחים את עץ החיפוש שכבה אחרי שכבה עד שמוצאים מצב מטרה. עובדים עם תור FIFO.

- הגדרות b - Branching factor. מקסימום ילדים של קודקוד
- d - Goal depth

האלגוריתם שלם אם $b < \infty$, וגם אופטימלי אם המחיר אחיד.	
האלגוריתם כבד - $\Theta(b^{d+1})$ זמן ו- $\Theta(b^d)$ מקום.	
DFS	Depth First Search. כמו BFS, אבל משתמש במחסנית LIFO במקום תור FIFO. גם אין בו זיהוי חזרות
הגדרה: m - עומק החיפוש המקסימלי. כמה אפשר ללכת בלי למצוא מטרה. לא בהכרח סופי.	
בניגוד לBFS, שהוא שלם כל עוד $b < \infty$, DFS יהיה שלם אם ורק אם בנוסף לכך גם אין מעגלים.	
DFS אינו אופטימלי. הוא יכול לבקר יעד במחיר גבוה יותר	
סיבוכיות הזמן שלו היא $\Theta(b^m)$, אבל סיבוכיות המקום שלו מאוד טובה - $\Theta(b \cdot m)$.	
DFS-L	Depth-Limited DFS. וואריאנט של DFS, אבל מגביל את עומק החיפוש ל- L . אם מגיעים לקודקוד בעומק L - לא מפתחים.
האלגוריתם לא שלם. הגבול הוא קלט - אם $L < d$ הוא לא יוכל להגיע למטרה. הוא גם לא אופטימלי.	
IDDFS	Iterative Deepening DFS. משתמש ב-DFS-L. מתחיל מ- $L = 0$, ואם לא מוצאים עושים $L \leftarrow L + 1$ ומנסים שוב - עד שמוצאים.
האלגוריתם שלם, כי בשלב מסויים מגיעים ל- $L = d$.	
זמן ריצה - $\Theta(b^d)$. סיבוכיות מקום - $\Theta(b \cdot d)$.	
UCS	Uniform-Cost Search. רוצים מחיר מינימלי כאשר המחירים שלנו לא אחידים (אבל גם לא שליליים)
מחזיקים תור עדיפויות (באמצעות ערימה) לפי פונקציית המחיר $g(s)$ - כלומר בראש הערימה יהיה הקודקוד הכי זול.	
האלגוריתם שלם (אם יש מעגלים ויש פעולות שמחירן 0, צריך זיהוי חזרות)	
האלגוריתם אופטימלי. ליעד הראשון שמוציאים מהערימה יש מחיר מינימלי.	
בכל האלגוריתמים מנגנון זיהוי חזרות עוזר - אבל לא לסיבוכיות, כי במקרה הכי גרוע אין חזרות.	

אלגוריתמים לוקאליים

נסתכל על אלגוריתם Random Walk - בכל שלב הוא בוחר מצב בן אקראי. האלגוריתם לא ימצא פתרון, אבל אם יש הרבה מסלולים שיכולים להביא לפתרון הוא ימצא בסבירות טובה. האלגוריתם מאוד חסכני בזיכרון, כי הוא צריך לשמור רק את המסלול.